

Designing an LLM-Tool Trading Research System That Learns From Its Own Tool Use

What a system like this can and cannot do

A blunt reality check: consistently extracting tradable edge from public markets is hard because competition is intense and information gets incorporated quickly. The classic “efficient markets” view argues that prices reflect available information, implying persistent outperformance is difficult without taking additional risk (and even then, outperformance is not guaranteed). ¹

If you aim to automate trading decisions, you’re operating in a domain where many retail participants lose money. The Securities and Exchange Commission ² has published investor education noting that day traders “typically suffer severe financial losses” early and that many never become profitable. ³

LLMs **can** materially improve your ability to *gather, normalize, and reason over messy, unstructured inputs* (filings, news, transcripts, social chatter, macro releases) and to *run consistent research loops*—but that is not the same thing as “finding an exploitable trend.” A warning example from the research literature: a 2025 paper presented at NeurIPS ⁴ reports a fine-tuned agent that achieved *higher* market-state classification accuracy yet produced *lower* simulated trading returns, attributing the gap to objective mismatch (optimizing a proxy task that doesn’t align with profitability). ⁵

So it helps to frame the goal precisely:

- **Feasible, high-leverage goal:** build a system that reliably turns heterogeneous information into timestamped, queryable artifacts and evaluation signals (what it fetched, why, what it concluded, what later happened). ⁶
- **Hard, end-to-end goal:** autonomously trade profitably and robustly across regimes, with controlled risk and without overfitting. Surveys of LLM-based trading agents emphasize that many published results are still constrained by backtesting shortcuts, data leakage risks, and fragile evaluation setups. ⁷

A practical division of labor that shows up both in industry workflows and research is: use LLMs primarily for **tool orchestration + unstructured-data understanding**, while using conventional quantitative methods for **signal construction, risk controls, and execution** (where determinism and auditability matter). ⁸

Information sources and tool modalities worth supporting

Your tool suite (web search, social search, URL fetch, plus market/fundamental data APIs) can be thought of as producing two broad “data shapes,” and your architecture should treat them differently:

Structured time series / records (prices, volumes, fundamentals tables, economic series) are best stored as normalized tables with precise timestamps and corporate-action adjustments. ⁹

Unstructured text streams (articles, filings text, posts) are best stored as immutable documents + extracted annotations (entities, events, sentiment, claims), with origin metadata and retrieval scores. ¹⁰

Below are the highest-signal categories (not “alpha,” but high-signal *inputs*) that systems like yours usually ingest.

Market data (quotes/trades/bars) Real-time and historical market data APIs give you OHLCV bars, sometimes ticks, and sometimes WebSocket streaming. Examples include Alpha Vantage ¹¹ (intraday OHLCV), Tiingo ¹² (IEX-based real-time endpoints), and Polygon.io ¹³ (WebSocket streams for trades/aggregates). ¹⁴

If you care about *true* real-time behavior, pay attention to exchange licensing, feed type (SIP vs proprietary), and whether you’re consuming a broker’s consolidated feed vs a single venue. Even venue-specific feeds are described as proprietary “real-time feeds” by the IEX Exchange ¹⁵ . ¹⁶

Regulatory filings and fundamentals For U.S. issuers, filings are one of the cleanest “truth sources” because they’re primary disclosures. The SEC’s EDGAR ¹⁷ APIs provide JSON access to submission history and XBRL financial statement data (including forms such as 10-K, 10-Q, and 8-K), and the SEC notes these JSON structures are updated throughout the day as submissions come in and do not require API keys. ¹⁸

This kind of source is ideal for an “exploration & self-review” loop because: - the documents are time-stamped and immutable (good for reproducibility), - extraction quality is measurable (did you detect the right event?), - and it’s harder to manipulate than social content. ¹⁹

Macro and rates data For macro context and regime features, Federal Reserve Bank of St. Louis ²⁰ provides the FRED ²¹ API for economic time series and related metadata. ²²

Macro data is where timestamps, revisions, and “vintage” handling matter if you want realistic backtests (what was known when). Even third-party wrappers emphasize point-in-time handling via ALFRED vintages. ²³

News and global events If you want a robust alternative to scraping random sites, the GDELT Project ²⁴ offers a large-scale event and news-derived dataset described as a near-real-time “global graph” of society via news media, with extensive documentation and bulk download options. ²⁵

This is attractive for systematic experimentation because you can standardize queries and compare extraction methods over a consistent corpus. ²⁶

Social / crowd content (treat as adversarial) If you ingest social content for “sentiment” or narrative detection, treat it as *hostile terrain*. The SEC explicitly warns that fraudsters can manipulate share prices by spreading rumors on social media and profit at investors’ expense. ²⁷

The platform APIs also impose practical constraints. The X ²⁸ API’s “recent search” endpoint is described as retrieving posts from the last 7 days, and the platform documents rate limits as central constraints on how many requests you can make per time window. ²⁹

This matters for your “learning to use tools better” goal: the optimal strategy may be *not* to over-invest in a source whose coverage window and rate limits force you into sampling artifacts rather than true coverage.

30

Architecture choices: agentic loop vs deterministic pipelines

A useful way to think about “agents” is: they are tool-using loops that can decide *what to do next*. A “pipeline” is a mostly predetermined graph of steps that runs in a fixed order.

31

You do not need a fully general agent to get the “explore → log → review → improve” behavior you described. But you **do** need (a) multi-step tool calling, (b) state, and (c) evaluation signals. Those can be implemented either as: - a **workflow orchestrator** (fixed DAG with occasional branches), or - an **agentic controller** (dynamic planning, repeated tool calls), or - a hybrid (which is what most production systems converge on).

32

Tool calling as the backbone At the API level, modern “tool calling” is typically implemented as a multi-step exchange: you provide tool schemas, the model emits a tool call with arguments, your code executes it, then you feed the tool output back for the model to incorporate.

33

This pattern is explicitly described (including vocabulary like “tools,” “tool calls,” tool outputs, and a multi-step flow) in OpenAI’s function/tool calling documentation.

34

For orchestrating many steps and retaining full provenance, a first-class tracing system matters more than whether you label something “agent” or “pipeline.” The OpenAI Agents SDK explicitly positions agentic apps as tool-using systems with handoffs and “a full trace of what happened.”

35

Agent-like reasoning patterns that work well for exploration A dominant pattern for tool-using reasoning is “think → act → observe → think...”, popularized by ReAct (“Reasoning and Acting”), which combines intermediate reasoning traces with tool actions.

36

For your use-case, the big win of this pattern is not philosophical. It’s operational: you can log each action/observation pair and later evaluate whether a tool call was useful, redundant, or misleading.

37

Tool discovery and “acquiring new tools” Your idea of “learning what tools are worth it” becomes more scalable if you formalize tools behind a standard interface that supports discovery (list tools, get schemas, call tools). The Model Context Protocol (MCP)—introduced publicly by Anthropic as an open standard for connecting models to external data/tools—formalizes servers exposing tools with names and schemas.

38

On the OpenAI side, OpenAI’s connectors/MCP documentation describes a concrete discovery step: when you specify a remote MCP server, the API attempts to list tools from the server and returns an output item containing imported tools.

39

This kind of mechanism is how “tool acquisition” can be made real: your system can enumerate tools, store their schemas/docs, and then (crucially) evaluate tool utility over time using the same logging and scoring machinery you apply to web and market data.

40

Where agents help, and where they hurt For trading research, agents are most useful in these roles: - **Hypothesis generation and refresh** (what changed? what to test next?), - **Contextual retrieval plans** (what filings/news/macro series should be fetched given a question), - **Data QA and reconciliation** (why do two sources disagree?), - **Postmortems** (why did this run waste tokens/time; what should be pruned?). ⁴³

Agents are most dangerous when you let them: - directly execute trades without hard limits, - browse untrusted content with write-capable tools, - or “optimize” on short-horizon backtests without strong leakage controls. ⁴⁴

Traceability and storage: building a real “flight recorder”

Your logging requirement (timestamped data + full traces showing which tool calls produced it) is not optional if you want sustainable improvement. Without a high-fidelity record, “self-review” degenerates into vibes.

Distributed tracing standards map well to LLM systems A standard way to represent multi-step execution is a **trace** made of **spans** (each span is one unit of work, with timestamps and attributes). OpenTelemetry ⁴⁵ documents traces/spans and how exporters send traces to backends (console, collectors, vendor systems). ⁴⁶

This structure matches your needs because you can model: - one top-level trace per “research run,” - child spans for each model call, - child spans for each tool call (web query, X search query, filings fetch, price history call), - plus derived spans for parsing, feature extraction, and evaluation. ⁴⁷

LLM-focused observability backends already speak “trace” Several modern LLM observability products explicitly structure runs as traces/spans and ingest via OpenTelemetry protocols:

- Arize Phoenix ⁴⁸ describes tracing as capturing a run step-by-step (model calls, retrieval, tool use, custom logic) and notes OTLP ingestion and auto-instrumentation for common frameworks. ⁴⁹
- Langfuse ⁵⁰ describes capturing traces with nested observations including timing, inputs, outputs, and cost. ⁵¹
- OpenAI’s cookbook example on evaluating agents with Langfuse ⁵⁰ describes traces containing spans for agent runs, tool calls, and model calls. ⁵²
- LangSmith ⁵³ positions itself as tracing + evaluation, including online evaluation of real user interactions. ⁵⁴

You can choose any backend; the important part is: **store your raw artifacts separately from your trace metadata** so you can re-run parsing and feature extraction later without re-fetching sources (and so your provenance stays intact). ⁵⁵

A concrete storage pattern that stays sane A sustainable design is an **append-only event log** (durable, immutable) plus derived “materialized views” (fast query tables). The immutable event log is the only thing you truly trust; everything else can be recomputed.

At minimum, store these event types:

- **ToolCallRequested**: tool name, normalized arguments, parent span id

- **ToolCallReturned**: raw output blob (or pointer), latency, bytes, parse status
- **DocIngested**: canonical URL/id, fetch timestamp, content hash
- **ExtractionProduced**: extracted entities/events/signals + confidence + versioned extractor id
- **DecisionProposed**: (if you do trading) proposed trade, rationale, constraints, risk checks
- **DecisionExecuted / Rejected**: execution outcome and why
- **OutcomeObserved**: later realized return, drawdown contribution, slippage, etc. 56

Here’s a minimal JSON-style schema for a single event (illustrative, not a standard):

```
{
  "event_id": "uuid",
  "run_id": "uuid",
  "trace_id": "otel_trace_id",
  "span_id": "otel_span_id",
  "parent_span_id": "otel_parent_span_id",
  "ts_utc": "2026-02-11T15:04:05.123Z",
  "event_type": "tool_call_returned",
  "tool_name": "edgar_submissions",
  "tool_args": {"cik": "0000320193"},
  "artifact_ref": "s3://bucket/raw/....json",
  "content_hash": "sha256:...",
  "latency_ms": 842,
  "token_cost": {"prompt": 0, "completion": 0},
  "status": "ok",
  "error": null,
  "tags": {"asset": "AAPL", "task": "filings_refresh"}
}
```

If you adopt OpenTelemetry IDs, you get compatibility with a large ecosystem of tracing and correlation tools (and you can still keep your own domain IDs and domain-specific tables). 57

What “lessons learned” look like in modern agent research

A key point: most “self-improving agent” work is **not** about the model updating its weights online. It’s about the system updating *text, memory, retrieval indices, prompts, or tool-use policies* between episodes. Multiple well-cited lines of research converge on this.

Reflection as stored text that changes future behavior The “Reflexion” framework explicitly proposes improving a language agent not by weight updates but by producing **verbal reflections** from feedback signals and storing them in an **episodic memory buffer** to guide later trials. 58

This is directly aligned with your goal: “lessons” can be stored as structured natural language objects (plus metadata), and later retrieved when similar situations arise.

“Experience logs → higher-level reflections” In “Generative Agents,” the proposed architecture stores a complete record of an agent’s experiences in natural language, synthesizes them into higher-level reflections, and retrieves them to plan behavior. ⁵⁹

Even though that work is about simulated people, the lesson-transfer is straightforward: your trading research system can store raw “episodes,” then periodically distill them into more compact “what worked / what didn’t” summaries keyed to contexts (market regime, tool type, asset class, task). ⁶⁰

Procedural knowledge as a “skill library” “Voyager” (lifelong learning in Minecraft) is notable because it frames learned behavior as an explicit **skill library** that accumulates reusable procedures the agent can call for new tasks. ⁶¹

In a trading research context, “skills” often end up being: - parameterized query templates (“fetch latest 8-Ks for ticker X; extract items Y”), - parsing/extraction code modules, - or multi-step retrieval recipes (“macro shock check: pull CPI release; compare expectations; map sector sensitivity”). ⁶²

This is where “skills.md” (or similar) can work—but **you should treat it as a compiled artifact**, not a dumping ground. If it grows without structure, it becomes another prompt soup.

Long-term memory as a managed resource, not a growing context window “MemGPT” frames memory as a hierarchy of tiers and uses mechanisms akin to paging/interrupts to manage limited context. ⁶³

This is a direct answer to your “don’t become bloated” constraint: the sustainable form of “learned lessons” is not “keep everything in the prompt,” but rather “keep everything in storage, and learn retrieval + distillation policies.” ⁶⁴

Tool-use competence can be trained, benchmarked, and stabilized Several research threads focus specifically on models learning to call tools:

- “Toolformer” proposes that LMs can teach themselves to use tools via self-supervision, learning when to call APIs and how to integrate outputs. ⁶⁵
- “ToolLLM” and related work introduce large-scale datasets/benchmarks for tool use (ToolBench) and discuss framework elements for training and evaluation. ⁶⁶
- “API-Bank” provides an executable evaluation system and training data for tool-augmented dialogues, explicitly treating planning/retrieval/tool calling as measurable competencies. ⁶⁷
- “ToolBench” positions itself as an open platform for training/serving/evaluating tool learning, and “StableToolBench” discusses stabilizing evaluation with caching and simulators because APIs change over time—this is extremely relevant to your “save all data with traces” requirement. ⁶⁸
- “Gorilla” argues that retrieval over API documentation + training can reduce hallucinated tool usage and adapt to changing docs (retrieval-aware training). ⁶⁹

Put differently: “learning to use tools better” is not only possible—it’s one of the most actively explored directions in agent research—but the best-performing systems typically combine **(a) retrieval over tool docs/schemas, (b) supervised or synthetic training data, and (c) rigorous tool-call evaluation harnesses.** ⁷⁰

Improving over time without turning into a mess

If you want sustained improvement, you need two things that many agent demos lack: (1) a stable target behavior you can measure, and (2) a controlled mechanism for updating the system based on those measurements. There are several proven mechanisms, each with tradeoffs.

Iterative self-critique loops (no weight updates) “Self-Refine” shows an iterative loop where the model produces an output, critiques it, and refines it repeatedly—without additional training data or reinforcement learning. ⁷¹

In your context, Self-Refine-like loops are most valuable for: - improving query plans (“what should I fetch next?”), - improving extraction instructions (“how to parse this filing?”), - and improving report quality (“what evidence supports this claim?”). ⁷²

They are **not** a substitute for quantitative evaluation of trading outcomes.

Prompt and instruction optimization (treat prompts as parameters) OPRO (“LLMs as optimizers”) frames optimization as repeatedly proposing new candidate instructions/prompts conditioned on prior candidates and their scores—explicitly applying the idea to prompt optimization. ⁷³

Promptbreeder similarly treats prompts as evolving artifacts, using an evolutionary process that mutates and selects prompts (including “mutation prompts” that guide how mutations happen). ⁷⁴

These methods are powerful if—and only if—you have a robust evaluation set and you can prevent reward hacking (for trading, that means careful out-of-sample and forward testing, not just optimizing on a backtest). ⁷⁵

Declarative “self-improving pipelines” instead of ad-hoc prompts DSPy ⁷⁶ explicitly positions itself as “declarative self-improving” Python modules, aiming to make LLM programs more reliable and maintainable than hand-managed prompting. ⁷⁷

This approach can map well to your use-case: you can express retrieval, extraction, and synthesis as modules, then use optimization/search methods to tune module instructions based on logged performance. ⁷⁸

Evaluation-driven development for agents To prevent “learning” from degenerating into story-telling, you need evaluators that can run automatically on logged traces.

- TruLens ⁷⁹ describes “feedback functions” as a programmatic way to generate evaluations on an application run (often using another model as an evaluator). ⁸⁰
- Dentity[["organization", "Weights & Biases", "ml ops company"]]’s Weave docs describe evaluation-driven LLM development centered on an `Evaluation` object that measures behavior over curated examples. ⁸¹
- LangSmith ⁵³ similarly emphasizes tracing + evaluation, including online evaluation on real traffic.

⁸²

You can apply this by defining multiple “scores,” not only PnL: - tool usefulness (did this call add novel information?), - source credibility (is this primary vs secondary; is it known-scammy), - extraction correctness (did it correctly classify/locate the relevant filing section), - latency and cost, - and only downstream: predictive value / trading utility. ⁸³

Memory compaction and “forgetting” as a first-class policy The highest-quality way to avoid bloat is to treat memory as: 1) immutable raw logs, and
2) a compact, actively maintained “working set” derived from those logs.

“MemGPT” is essentially a formalization of this idea: manage multiple memory tiers so the model only sees what it needs, when it needs it. ⁶³

In practice, this means your “lessons” store should support: - versioning (lessons can be superseded), - confidence and decay (lessons can expire), - and retrieval gating (only inject lessons that match the current task/regime). ⁶⁴

Evaluation, safety, and the non-negotiable “reality checks” for markets

Backtests are where most “self-improving trading systems” fool themselves. Your loop will tend to optimize whatever feedback you make easy to measure—so make sure the feedback is not a trap.

Backtest overfitting is common and quantifiable Bailey and López de Prado’s work on the “probability of backtest overfitting” proposes a framework (including combinatorially symmetric cross-validation) to estimate how likely a chosen strategy is overfit in the context of investment simulations. ⁸⁴

The CFA Institute ⁸⁵ also highlights overfitting as a major backtesting risk: a strategy can look exceptional on historical data and fail in new regimes. ⁸⁶

If you introduce an LLM into the loop, you add new overfitting surfaces: - it can meta-optimize toward quirks of your evaluation set, - it can exploit leakage in your retrieval process, - and it can find shortcuts that improve a proxy score but not real outcomes (exactly the failure mode described in the NeurIPS 2025 “predicts but loses” case). ⁸⁷

Algorithmic trading requires controls, even in “research” mode Even if you are not a broker-dealer, it’s smart to borrow the control philosophy from regulated environments.

FINRA ⁸⁸ summarizes the SEC’s Market Access Rule (Exchange Act Rule 15c3-5) as requiring firms with market access to control risks associated with market access to protect market integrity and stability. ⁸⁹ The rule itself (as codified) describes obligations around risk management controls and supervisory procedures designed to manage financial/regulatory risks of market access. ⁹⁰ SEC staff FAQs also discuss pre-order risk management controls under this rule. ⁹¹

Translated into your system design: if the system ever progresses from “research” to “execution,” you want hard guardrails that an LLM cannot talk its way around: max order sizes, kill switches, allowed instruments, and independent risk checks. ⁹²

Social data is manipulable and sometimes criminal For any strategy that “spots trends” by watching social chatter, you have to confront manipulation directly. The SEC has published investor alerts describing rumor-based manipulation on social platforms. ²⁷

The SEC has also brought enforcement actions involving social-media-driven stock manipulation promoted on platforms like Twitter and Discord. ⁹³

This has a direct implication for your exploration loop: some of your “best performing” information sources may be best at producing *compelling narratives*, not reliable signals. Your reviewer model (or self-review stage) needs explicit checks for source credibility and manipulation indicators. ⁹⁴

Tool-using agents have a security problem: prompt injection Once you combine web browsing + tool calls (especially write-capable tools), you inherit the prompt-injection threat class. OWASP ⁹⁵’s prompt injection prevention cheat sheet describes prompt injection as a vulnerability where malicious input manipulates model behavior, in part because instructions and data are processed together. ⁹⁶

If you also adopt MCP-style tool ecosystems, security guidance becomes even more important. MCP documentation includes explicit security best practices discussing risks like “expanded blast radius” from overly broad tokens/scopes. ⁹⁷

For a trading-adjacent system, the sober takeaway is: **never let a model directly execute privileged actions based on untrusted text** (webpages, posts, emails). Separate: - browse/fetch (untrusted), - parse/extract (sanitized), - decide (policy-checked), - execute (guardrailed). ⁹⁸

A final “tell it like it is” summary What you are proposing is realistic if you define “learning” as **system-level improvement**: better retrieval recipes, better tool selection, better parsing/extraction, better evaluation discipline, and better filtering of bad sources. Research frameworks like Reflexion, Generative Agents, Voyager, and MemGPT all embody this idea in different forms (reflections, experience logs, skill libraries, hierarchical memory). ⁹⁹

But if the hidden goal is “let the model browse the web and discover a consistently exploitable market pattern,” you should expect the dominant failure mode to be **overfitting + narrative seduction** unless evaluation is brutal, leakage-resistant, and grounded in realistic forward testing and risk controls. The published case where an agent predicts better but earns less is not an edge case; it’s the warning label. ¹⁰⁰

¹ ³⁵ Efficient Capital Markets: A Review of Theory and Empirical ...

https://people.hec.edu/rosu/wp-content/uploads/sites/43/2023/09/Fama-Efficient-capital-markets-1970.pdf?utm_source=chatgpt.com

² ²⁹ ³⁰ Search recent Posts

https://docs.x.com/x-api/posts/search-recent-posts?utm_source=chatgpt.com

³ Day Trading: Your Dollars at Risk

https://www.sec.gov/about/reports-publications/investorpubsdaytipshtm?utm_source=chatgpt.com

⁴ ²⁵ ³⁹ The GDELT Project

https://www.gdeltproject.org/?utm_source=chatgpt.com

- 5 87 100 **NeurIPS The Losing Winner: An LLM Agent That Predicts the Market but Loses Money**
<https://neurips.cc/virtual/2025/132549>
- 6 11 38 46 47 56 57 **Traces**
https://opentelemetry.io/docs/concepts/signals/traces/?utm_source=chatgpt.com
- 7 43 45 **Large Language Model Agent in Financial Trading: A Survey**
<https://ideas.repec.org/p/arx/papers/2408.06361.html>
- 8 79 **Unstructured Data and AI**
https://rpc.cfainstitute.org/sites/default/files/-/media/documents/article/industry-research/unstructured-data-and-ai.pdf?utm_source=chatgpt.com
- 9 14 **Alpha Vantage API Documentation**
https://www.alphavantage.co/documentation/?utm_source=chatgpt.com
- 10 18 19 28 50 55 62 72 88 **EDGAR Application Programming Interfaces (APIs)**
https://www.sec.gov/search-filings/edgar-application-programming-interfaces?utm_source=chatgpt.com
- 12 66 **ToolLLM: Facilitating Large Language Models to Master ...**
https://arxiv.org/abs/2307.16789?utm_source=chatgpt.com
- 13 71 **Self-Refine: Iterative Refinement with Self-Feedback**
https://arxiv.org/abs/2303.17651?utm_source=chatgpt.com
- 15 27 85 94 **Social Media and Stock Tip Scams – Investor Alert**
https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins/social-media-stock-scams?utm_source=chatgpt.com
- 16 **Market Data & Connectivity | IEX Exchange**
https://www.iexexchange.io/products/market-data-connectivity?utm_source=chatgpt.com
- 17 84 **The Probability of Backtest Overfitting**
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2326253&utm_source=chatgpt.com
- 20 34 58 99 **Reflexion: Language Agents with Verbal Reinforcement Learning**
https://arxiv.org/abs/2303.11366?utm_source=chatgpt.com
- 21 80 83 **Feedback Functions**
https://www.trulens.org/getting_started/core_concepts/feedback_functions/?utm_source=chatgpt.com
- 22 **St. Louis Fed Web Services: FRED® API**
https://fred.stlouisfed.org/docs/api/fred/?utm_source=chatgpt.com
- 23 **mortada/fredapi: Python API for FRED (Federal Reserve ...**
https://github.com/mortada/fredapi?utm_source=chatgpt.com
- 24 31 32 **Workflows and agents - Docs by LangChain**
https://docs.langchain.com/oss/python/langgraph/workflows-agents?utm_source=chatgpt.com
- 26 **Data: Querying, Analyzing and Downloading**
https://www.gdeltproject.org/data.html?utm_source=chatgpt.com
- 33 **Function calling | OpenAI API**
<https://platform.openai.com/docs/guides/function-calling>

36 Agents SDK | OpenAI API

<https://platform.openai.com/docs/guides/agents-sdk>

37 Synergizing Reasoning and Acting in Language Models

https://huggingface.co/papers/2210.03629?utm_source=chatgpt.com

40 Introducing the Model Context Protocol

https://www.anthropic.com/news/model-context-protocol?utm_source=chatgpt.com

41 42 Connectors and MCP servers | OpenAI API

https://developers.openai.com/api/docs/guides/tools-connectors-mcp/?utm_source=chatgpt.com

44 75 95 THE PROBABILITY OF BACKTEST OVERFITTING

https://www.davidhbailey.com/dhbpapers/backtest-prob.pdf?utm_source=chatgpt.com

48 49 What is Arize Phoenix?

https://arize.com/docs/phoenix?utm_source=chatgpt.com

51 LLM Observability & Application Tracing (open source)

https://langfuse.com/docs/observability/overview?utm_source=chatgpt.com

52 Evaluating Agents with Langfuse

https://developers.openai.com/cookbook/examples/agents_sdk/evaluate_agents/?utm_source=chatgpt.com

53 93 SEC Charges Eight Social Media Influencers in \$100 ...

https://www.sec.gov/newsroom/press-releases/2022-221?utm_source=chatgpt.com

54 LangSmith docs - Docs by LangChain

https://docs.langchain.com/langsmith/home?utm_source=chatgpt.com

59 60 Generative Agents: Interactive Simulacra of Human Behavior

https://arxiv.org/abs/2304.03442?utm_source=chatgpt.com

61 Voyager: An Open-Ended Embodied Agent with Large ...

https://arxiv.org/abs/2305.16291?utm_source=chatgpt.com

63 64 MemGPT: Towards LLMs as Operating Systems

https://arxiv.org/abs/2310.08560?utm_source=chatgpt.com

65 Toolformer: Language Models Can Teach Themselves to Use Tools

https://arxiv.org/abs/2302.04761?utm_source=chatgpt.com

67 API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs

https://arxiv.org/abs/2304.08244?utm_source=chatgpt.com

68 OpenBMB/ToolBench

https://github.com/OpenBMB/ToolBench?utm_source=chatgpt.com

69 70 Gorilla: Large Language Model Connected with Massive APIs

https://arxiv.org/abs/2305.15334?utm_source=chatgpt.com

73 Large Language Models as Optimizers

https://arxiv.org/abs/2309.03409?utm_source=chatgpt.com

74 Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution

https://arxiv.org/abs/2309.16797?utm_source=chatgpt.com

76 91 **Trading & Markets Frequently Asked Questions**

https://www.sec.gov/rules-regulations/staff-guidance/trading-markets-frequently-asked-questions/divisionsmarketregfaq-0?utm_source=chatgpt.com

77 78 **DSPy**

https://dspy.ai/?utm_source=chatgpt.com

81 **Evaluations overview**

https://docs.wandb.ai/weave/guides/core-types/evaluations?utm_source=chatgpt.com

82 **LangSmith Evaluation - Docs by LangChain**

https://docs.langchain.com/langsmith/evaluation?utm_source=chatgpt.com

86 **Investment Model Validation**

https://rpc.cfainstitute.org/sites/default/files/-/media/documents/article/rf-brief/investment-model-validation.pdf?utm_source=chatgpt.com

89 92 **Market Access Rule**

https://www.finra.org/rules-guidance/guidance/reports/2022-finras-examination-and-risk-monitoring-program/market-access-rule?utm_source=chatgpt.com

90 **17 CFR § 240.15c3-5 - Risk management controls for brokers ...**

https://www.law.cornell.edu/cfr/text/17/240.15c3-5?utm_source=chatgpt.com

96 98 **LLM Prompt Injection Prevention Cheat Sheet**

https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html?utm_source=chatgpt.com

97 **Security Best Practices**

https://modelcontextprotocol.io/specification/draft/basic/security_best_practices?utm_source=chatgpt.com